부산대학교
PUSAN NATIONAL UNIVERSITY

AVL Balance

Keeping the Tree Short

Height Balance and the Balance Factor

DS3600082-140
자료구조
2026학년도 2학기

오용경
데이터사이언스전공
yongkyung.oh@pusan.ac.kr

Section 1 • AVL Balance

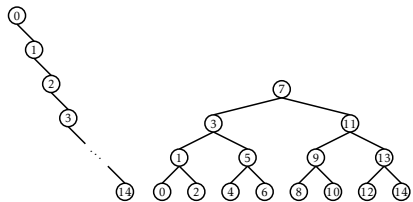
The Skewed-Tree Problem

BST가 최악으로 기울 때 로그 시간 보장이 사라지는 이유를 확인한다

BST의 높이가 탐색 비용을 정한다

탐색은 루트에서 내려가므로 높이만큼의 비교가 든다

트리의 모양	높이	탐색 비용
고르게 퍼진 트리	약 $\log_2 n$	$O(\log n)$
한쪽으로 기운 트리	$n - 1$	$O(n)$



0부터 14까지를 차례로 넣으면 왼쪽(높이 14), 7, 3, 11처럼 가운데부터 넣으면 오른쪽(높이 3)이 된다. 14를 찾는 비교는 15번과 4번이다.

Source: Morin (2013), Figure 7.1.

정렬된 값을 넣으면

삽입 순서	높이	모양	80을 찾는 비교 수
$50, 30, 70, 20, 40, 60, 80$	2	고르게 퍼짐	3
$20, 30, 40, 50, 60, 70, 80$	6	한 줄	7

> 같은 키라도 편향 트리(skewed tree)가 되면 탐색이 연결리스트처럼 $O(n)$이 든다

삽입 순서는 사용하는 쪽이 정하지 못한다

상황	결과와 필요한 조치
이미 정렬된 값을 삽입	한쪽으로 기울어 높이가 $n-1$이 된다
입력을 미리 섞을 수 없음	삽입할 때 트리의 균형을 유지한다
보장할 목표	입력 순서와 무관한 $O(\log n)$ 높이

> 삽입 순서를 통제할 수 없으므로 최악의 경우 높이를 제한해야 한다

균형 트리

정의

균형 트리(balanced tree)

삽입·삭제 뒤에도 높이를 $O(\log n)$으로 유지하는 트리다.
삽입 순서 때문에 한쪽으로 길어지면 구조를 조정해 높이가 선형으로 커지는 것을 막는다.

균형 유지에 필요한 추가 작업

노드에 균형 정보를 저장하고 삽입 뒤에 갱신한다

항목	보통 BST	균형 트리
노드마다 저장	키와 자식 노드	균형 정보도 함께
삽입에서 하는 일	자리 찾기	자리 찾기와 재균형

Checkpoint

The Skewed-Tree Problem

- BST의 탐색 비용을 정하는 것은 무엇인가?
- 정렬된 값을 차례로 넣으면 어떤 모양이 되는가?
- 삽입 순서를 통제할 수 없을 때 무엇을 보장해야 하는가?

Section 2 • AVL Balance

Height Balance

균형을 재는 값을 정의하고 조건을 세운다

좌우 부분 트리의 높이 차이를 잰다

정의

균형 인수(balance factor)

한 노드의 왼쪽 부분 트리 높이에서 오른쪽 부분 트리 높이를 뺀 값이다.

$$bf(v) = h(v_{\text{left}}) - h(v_{\text{right}})$$

양수면 왼쪽이 더 높고, 음수면 오른쪽이 더 높다. 빈 부분 트리의 높이는 -1로 둔다.

노드마다 하나씩 값을 두면 어느 자리가 기울었는지 바로 알 수 있다

AVL 트리

정의

AVL 트리(AVL tree)

모든 노드에서 왼쪽과 오른쪽 부분 트리의 높이 차이가 1 이하인 BST다.
이 조건을 높이 균형(height balance)이라고 한다.

BST의 ordering invariant에 높이 조건 하나를 더 얹은 것이다

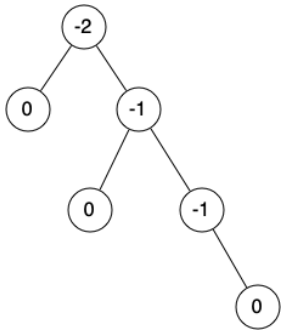
ordering invariant와 높이 균형

조건	정하는 것	탐색에 미치는 영향
ordering invariant	키의 자리	비교 결과로 왼쪽·오른쪽 선택
높이 균형	트리의 모양	탐색 경로의 길이를 제한

> 두 조건을 동시에 지켜야 AVL 트리다

기울어진 트리의 균형 인수

루트의 -2가 조건을 어긴 자리다



노드 안의 숫자가 균형 인수다. -1과 0은 AVL 조건을 지키고 -2는 어긴다.

Source: Miller and Ranum (2013), Balanced Binary Search Trees.

균형 인수의 부호가 무거운 쪽을 가리킨다

균형 인수	이름	뜻
> 0	왼쪽이 무거움(left-heavy)	왼쪽 부분 트리가 더 높다
< 0	오른쪽이 무거움(right-heavy)	오른쪽 부분 트리가 더 높다
0	같은 높이	좌우 부분 트리의 높이가 같다

좌우 높이가 정확히 같을 필요는 없다

조건	가능한 모양	노드 수의 제약
모든 노드에서 차이 0	모든 레벨이 가득 찬 트리	$n = 2^{h+1} - 1$만 가능
모든 노드에서 차이 1 이하	일부 레벨이 덜 차도 가능	임의의 n을 담을 수 있음
차이에 제한 없음	한쪽으로 기울 수도 있음	높이가 $n - 1$까지 증가

차이를 1까지 허용하므로 임의의 n개 노드를 담을 수 있다

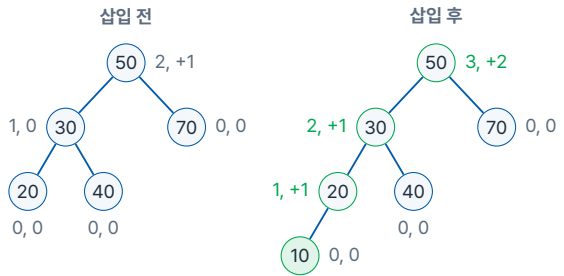
각 노드에 균형 정보나 높이를 저장한다

방법	비용과 저장하는 것
필요할 때마다 높이 계산	부분 트리 전체를 봐야 한다
균형 인수 저장	노드마다 균형 인수 하나
높이 저장	노드마다 부분 트리 높이 하나

> 높이나 균형 인수를 노드마다 저장하면 균형 확인이 $O(1)$이 된다

같은 트리에 10을 넣기 전과 후

먼저 50, 30, 70, 20, 40을 넣고 20의 왼쪽에 10을 넣는다. 노드 옆 값은 (높이, *bf*)다



노드	삽입 전	삽입 후
20	$0,\ 0$	$1,\ +1$
30	$1,\ 0$	$2,\ +1$
50	$2,\ +1$	$3,\ +2$
40, 70	$0,\ 0$	$0,\ 0$

> 루트만 보는 대신 경로 위 모든 조상의 값을 다시 계산해야 한다

Adapted from Miller and Ranum (2013), Balanced Binary Search Trees.

Checkpoint

Height Balance

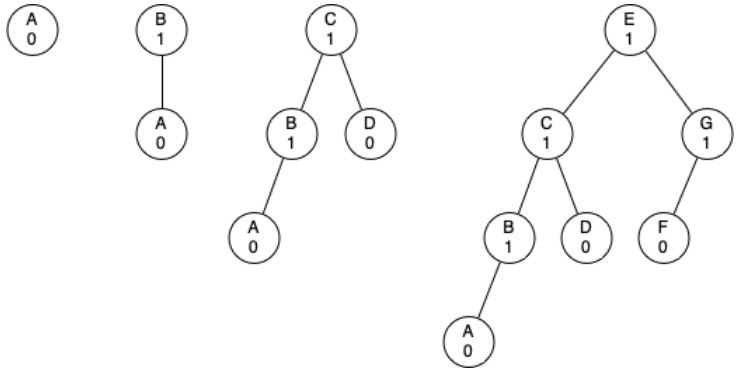
- 균형 인수는 어떻게 계산하는가?
- AVL 트리가 되기 위한 조건은 무엇인가?
- 단말 노드의 균형 인수가 0인 이유는 무엇인가?

Section 3 • AVL Balance

Why Balance Works

균형 조건이 왜 $O(\log n)$을 주는지 확인한다

가장 성긴 AVL 트리



높이 0, 1, 2, 3에서 노드를 가장 적게 담은 AVL 트리다. 각각 1, 2, 4, 7개다. 노드 아래 숫자는 균형 인수다. 높이 3 트리에서 C가 루트인 왼쪽은 높이 2, G가 루트인 오른쪽은 높이 1의 가장 성긴 트리다.

Source: Miller and Ranum (2013), AVL Tree Performance.

규칙을 식으로 적으면

한쪽은 $h-1$, 다른 쪽은 $h-2$인 가장 성긴 트리를 붙인 모양이다

항목	식
점화식	$N(h) = N(h-1) + N(h-2) + 1$
시작값	$N(0) = 1$, $N(1) = 2$

Adapted from UBC DSCI 221 (n.d.), AVL height-bound recurrence.

값을 늘어놓으면

h	**최소 노드 수**	$N(h)+1$
0, 1, 2, 3	1, 2, 4, 7	2, 3, 5, 8
4, 5, 6, 7	12, 20, 33, 54	13, 21, 34, 55

> $N(h)+1$ 열은 피보나치 수 2, 3, 5, 8, 13, 21, 34, 55와 같다

그래서 높이가 묶인다

트리	높이
n개를 담은 AVL 트리의 최대 높이	대략 $1.44 \log_2 n$
가장 균형 잡힌 트리	$h \approx \log_2 n$
두 높이의 비(점근적 근사)	약 1.44배

> 최소 노드 수가 높이에 대해 지수적으로 늘어나므로 $h = O(\log n)$이다

Source: Miller and Ranum (2013), AVL Tree Performance.

연산 비용이 따라온다

연산	BST	AVL 트리
탐색	$O(n)$ 최악	$O(\log n)$
삽입	$O(n)$ 최악	$O(\log n)$
삭제	$O(n)$ 최악	$O(\log n)$

탐색은 높이로 묶이고, 삽입·삭제는 재균형 비용을 더해도 $O(\log n)$이다

AVL 높이 상계와 편향 BST 높이

n	AVL 높이 상계	편향 BST
1,000	14	999
1,000,000	28	999,999

> 백만 개를 담아도 서른 번이 안 되는 비교로 어디든 닿는다

Checkpoint

Why Balance Works

- 높이가 h인 AVL 트리의 최소 노드 수는 어떤 점화식을 따르는가?
- 그 수가 지수적으로 커진다는 것이 왜 중요한가?
- 높이가 $O(\log n)$이면 탐색 비용은 어떻게 제한되는가?

Section 4 • AVL Balance

Detecting Imbalance

삽입 뒤 어디가 기울었는지 찾는 절차를 정한다

새 값이 들어오면

새 단말 노드가 생기면 그 위 조상들의 높이가 바뀔 수 있다

항목	내용
높이가 바뀔 수 있는 노드	새 단말 노드에서 루트까지의 경로 위
경로 밖의 노드	바뀌지 않는다
확인할 노드 수	높이만큼, 즉 $O(\log n)$개

올라가며 고치는 절차

단계	하는 일	보는 것
1	부모의 균형 인수를 고친다	올라온 방향(왼쪽 또는 오른쪽)
2	값이 0이 되면 멈춘다	높이가 안 늘었다
3	± 1이 되면 부모로 올라간다	높이가 1 늘었다
4	$\lvert bf \rvert = 2$면 재균형하고 멈춘다	재균형이 필요하다

> 균형 인수가 0이 되면 그 위 조상의 높이와 균형 인수는 바뀌지 않는다

AVL 노드에 부모와 균형 인수를 저장한다

필드	저장하는 것
`key`, `value`	탐색할 키와 그 키에 대응하는 값
`left_child`, `right_child`	왼쪽·오른쪽 자식 참조, 없으면 `None`
`parent`	부모 참조, 전체 트리의 루트에서는 `None`
`balance_factor`	저장된 $h(\text{left}) - h(\text{right})$
트리의 `_root`	트리 객체가 저장하는 전체 루트 참조

자식 참조는 BST의 `left`/`right`와 같은 역할이다. 새 단말 노드는 `parent`를 자신을 붙인 부모로, `balance_factor`를 0으로 둔다.

Adapted from Miller and Ranum (2013), Search Tree Implementation and AVL Tree Implementation.

부모 방향을 확인해 균형 갱신을 이어 간다

메서드	확인하거나 수행하는 일
`is_left_child()`	부모가 있고 부모의 왼쪽 자식이 자신인지 확인
`is_right_child()`	부모가 있고 부모의 오른쪽 자식이 자신인지 확인
`is_root()`	`parent`가 `None`인지 확인
`update_balance(node)`	삽입한 노드에서 부모로 올라가며 균형 인수 갱신
`rebalance(node)`	불균형 노드를 받아 높이 균형 복구

자식 방향은 키의 대소가 아니라 부모의 자식 참조로 판단한다. 왼쪽에서 높이가 늘면 부모의 균형 인수에 1을 더하고 오른쪽이면 1을 뺀다.

Adapted from Miller and Ranum (2013), Search Tree Implementation and AVL Tree Implementation.

update_balance가 올라가며 멈추는 곳

10을 넣은 뒤 update_balance(10)이 부모를 따라 한 칸씩 올라간다

```
1  def update_balance(self, node):
2      if node.balance_factor > 1 or node.balance_factor < -1:
3          self.rebalance(node)
4          return
5      if node.parent:
6          if node.is_left_child():
7              node.parent.balance_factor += 1
8          elif node.is_right_child():
9              node.parent.balance_factor -= 1
10         if node.parent.balance_factor != 0:
11             self.update_balance(node.parent)
```

node	줄	결과
10	6, 7, 10	20: $0 \to +1$
20	6, 7, 10	30: $0 \to +1$
30	6, 7, 10	50: $+1 \to +2$
50	2, 3, 4	재균형, 멈춤

balance_factor는 균형 인수 *bf*를 저장하고 parent는 부모를 가리킨다. 부모의 균형 인수가 0이 아니면 11행에서 올라가고, $|bf| > 1$이면 재균형한 뒤 멈춘다

Source: Miller and Ranum (2013), AVL Tree Implementation.

첫 불균형 조상은 50이다

새 값 10에서 부모를 따라 올라가며 균형 인수를 읽는다

확인 순서	*bf*	판정
10, 20	$0,\ +1$	두 노드 모두 균형
30	$+1$	아직 균형
50	$+2$	처음 조건을 어긴 조상

> 50에서 갱신을 멈추고 이 노드에서 재균형을 시작한다

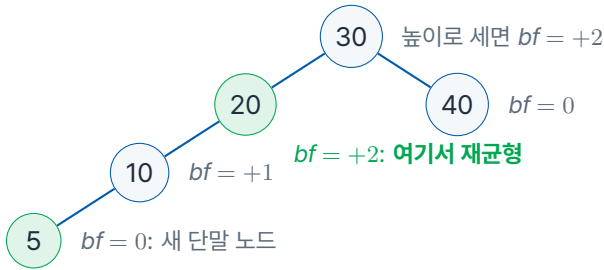
Adapted from Miller and Ranum (2013), AVL Tree Implementation.

새 값에 가장 가까운 불균형 노드를 고친다

항목	내용
$\lvert bf \rvert = 2$인 노드가 여럿일 때 그 노드를 고르는 이유	새 값에서 가장 가까운 노드를 고른다 그 아래는 아직 AVL 조건을 지킨다

삽입 뒤 처음 만난 불균형을 고치면 그 부분 트리는 삽입 전 높이로 돌아간다

첫 불균형 조상에서 재균형을 시작한다



5에서 부모를 따라 올라가면 10은 조건을 지키고, 20에서 처음 $|bf| = 2$가 된다. 20에서 재균형하고 멈추므로 30에 저장된 값은 다시 계산하지 않는다.

삽입 경로만 보면 되는 이유

노드	갱신에서의 역할
새 단말 노드의 조상이 아닌 노드	부분 트리 높이가 바뀌지 않는다
갱신 뒤 균형 인수가 0이 된 조상	그 부분 트리의 높이는 더 커지지 않는다
처음 $\lvert bf \rvert = 2$가 된 조상	그 노드에서 회전 방향을 판정한다

> 새 단말 노드에서 루트로 이어지는 경로만 갱신하면 된다

확인에 드는 비용

하는 일	비용	이유
한 노드 확인	$O(1)$	저장된 값을 읽는다
루트까지의 경로 전체	$O(\log n)$	높이만큼

Checkpoint

Detecting Imbalance

- 삽입 뒤 균형 인수를 고쳐야 하는 노드는 어디에 있는가?
- 갱신이 멈추는 조건은 무엇인가?
- $|bf| = 2$인 노드가 여럿이면 어느 것을 고르는가?

How Can a Rotation Restore Balance?

Next Lecture: AVL Rotations

- 회전이 키의 대소 관계를 유지하는 원리
- 한 번의 회전으로 균형을 회복하는 경우
- 두 번의 회전이 필요한 경우

오용경 yongkyung.oh@pusan.ac.kr